TatvaSoft

SOFTWARE DEVELOPMENT · GUIDELINE

TVS-DEV-AI-GL-01.6

Explore, Plan, Implement Modes

Claude • Cursor • GitHub Copilot • OpenAI Codex • Gemini | Project Standard

VERSION	EFFECTIVE DATE	PREPARED BY	APPROVED BY
1.0	**28.09.2026**	**Parthesh Mehta**	**Shital Patel**

HOW TO READ THIS DOCUMENT

Short on time? Read the **Core rule** and the **Stages and checks** table.

The **How each tool does it** shows how to do each stage in each AI tool.

Parent guideline: **TVS-DEV-AI-GL-01 AI Development Tools Guidelines** (Plan, Ask & Debug Mode section).

Document Control

Document Control Information

Document Title	AI DevTools – Plan / Ask / Debug
Document ID	TVS-DEV-AI-GL-01.6
Document Version	Version 1.0
Effective Date	28.09.2026
Parent Document	TVS-DEV-AI-GL-01 — AI Development Tools Guidelines

Document Publication History

Document Prepared By	Parthesh Mehta
Document Reviewed By	Jigar Prajapati
Document Approved By	Shital Patel
Initial Release Version	1.0
Initial Release Date	28.09.2026

Document Revision History

Version No.	Revision Description	Change Author	Change Approver	Revision Date
1.0	Final Release	Parthesh Mehta	Shital Patel	25.09.2026

Document Definition

This document sets a tool-neutral way of working with AI coding tools: Understand / Explore, Plan, Implement and Check / Review, with a human check at every stage, and Debug only when something fails. It shows how to do each stage in Claude Code, Cursor, GitHub Copilot and OpenAI Codex (plus Gemini CLI), and sets shared rules for context, model choice, data safety and review. Use it when starting a task, choosing how to work with AI or onboarding developers to a new tool.

Contents

Introduction	What this standard covers and when to use it
1. Core rule	Work by stage, check every stage, Debug only on failure
2. Stages and checks	Purpose, edits and check of each stage
3. How each tool does it	Claude Code · Cursor · Copilot · Codex · Gemini CLI
4. Explore	Rules for understanding the options
5. Plan	Rules for planning
6. Implement and Check / Review	Rules for making and checking changes
7. Debug	Rules for fixing failures
8. Common rules across all stages	Shared rules for every stage
9. Copy/paste prompt examples	Ready-to-use prompts by stage

Introduction

> **KEY TAKEAWAY**
>
> This document sets one way of working with AI coding tools: **Understand / Explore**, **Plan**, **Implement** and **Check / Review**, with a human check at every stage. **Debug** is used only when something fails.

The stages describe the purpose of the work, not a tool feature. Each tool names and supports them differently, so section 3 shows how to do each stage in **Claude Code, Cursor, GitHub Copilot and OpenAI Codex** (plus Gemini CLI).

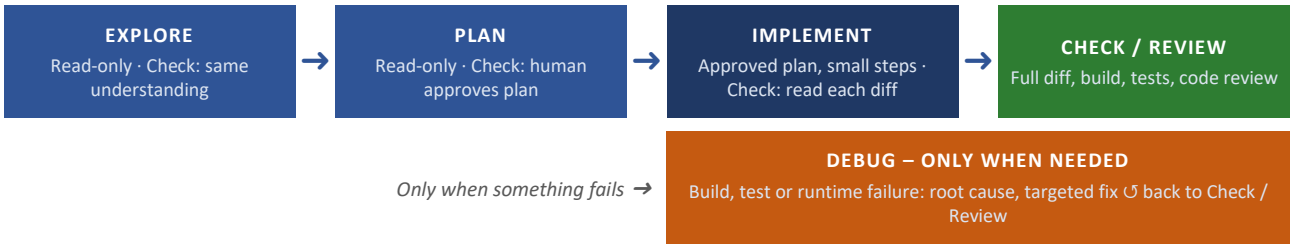
Use it when:

Starting a task	**Choosing how to work with AI**	**Onboarding developers to a new tool**

1. Core rule

> **CORE RULE**
>
> **Work by stage, not by tool feature: Understand / Explore → Plan → Implement → Check / Review.** Use what your tool provides for each stage (section 3), and move on only when the stage's check is done.
>
> Understand and Plan do not change code. **Debug only when something fails**, not after every change. **Every change is reviewed, built, and tested by the developer** before it is committed.

Typical flow. A human check closes every stage. Simple tasks can skip Plan: Understand → Implement → Check / Review.



2. Stages and checks

Stage	Use it for	Edits code?	Check before moving on
Explore / Understand	Exploring unfamiliar code, questions, comparing approaches, small suggestions	No	Developer confirms the AI has understood the problem and the code (same page)
Plan	Normal and complex tasks, refactors, unfamiliar code, work that needs sign-off	No; read-only research	**Human approves the plan**; tech lead for complex or high-impact work
Implement	Carrying out the approved plan in small steps	Yes	Developer reads each step's diff before the next step
Check / Review	Confirming the change is correct, safe and complete	No; fixes go back to Implement	Full diff reviewed, build and tests pass, then code review as usual
Debug (only when needed)	Build or test failures, exceptions, wrong behaviour, reported bugs	Yes, after you review	Root cause confirmed, fix reviewed, regression test passes; then back to Check / Review

3. How each tool does it

Tool	Understand / Explore	Plan	Implement	Debug
Claude Code	**Plan mode**, or Manual mode with each edit approved	Plan mode: **Shift+Tab** or `--permission-mode plan`	Leave Plan mode; Manual mode approves each edit	No separate mode: give repro steps and the stack trace
Cursor	**Ask**	**Plan**	**Agent**; review each diff	**Debug** mode (hypotheses + runtime logs)
GitHub Copilot – VS Code	**Ask** in the agent picker	**Plan** in the agent picker, or `/plan`	**Agent**, or hand the approved plan to Agent	**Agent** with the error, failing test or terminal output
GitHub Copilot – Visual Studio 2026	**Ask**	**Plan** agent (saves `.copilot/plans/`)	**Implement plan**, or **Agent** mode	**Debugger** agent, `@debugger`, or **Analyze with Copilot**
OpenAI Codex (CLI / IDE)	**Chat**, or `/permissions` → Read Only	`/plan`	`/permissions` → workspace edits; avoid Full Access	No separate mode: prompt with repro steps
Gemini CLI	**Plan mode**, or default mode with each edit approved	Plan mode: **Shift+Tab** (Default → Auto-Edit → Plan)	Default mode with each edit approved	No separate mode: prompt with repro steps

Check / Review works the same in every tool: read the diff in the tool's review view, build, run tests, then code review as usual.

Copilot: VS Code or Visual Studio. Prefer VS Code where the project allows; new Copilot features usually arrive there first. Use Visual Studio when the solution needs it (.NET Framework, WinForms/WPF designers, runtime debugging) and stay on the latest Visual Studio 2026 update. On Visual Studio 2022 there is no Plan agent: use Agent mode (17.14 or later) with "Create a plan first. Do not edit files until I approve."

Mode names and shortcuts change often. Check the tool's documentation (see references) if an option is missing.

4. Explore / Understand

- Use it to explore available options, unfamiliar code, ask questions and compare approaches. **It does not change code.**
- Give precise context: select the code, or reference files and folders with the tool's file-mention syntax.
- **Check:** confirm the AI's explanation matches the code and the requirement. Correct any wrong assumption before you plan.
- Do not use it as a development workflow. Every message re-sends the whole chat, so long Q&A chats cost more time and tokens. Move on to Plan, then Implement.
- Verify API details in official docs before you rely on them.

5. Plan

- Use Plan for **normal and complex** tasks; simple tasks can skip it.
- Give the requirement, constraints, and acceptance criteria up front, and answer the tool's clarifying questions.
- Review the plan for: files to change, order of steps, edge cases, validation, tests, and data migration or rollback where relevant.
- **Check:** a human approves the plan **before** any code changes; for complex work, the tech lead approves. Update the plan first if scope changes.

6. Implement and Check / Review

- Implement **only the approved plan**, in small steps. If scope changes, stop and update the plan first.
- Read each step's diff before the next step. Do not accept large changes you have not read.
- Use automatic edit approval only for low-risk work, never for security, authentication, production data or infrastructure changes.
- **Check / Review:** review the full diff, build, run tests, and go through code review as usual. Fixes go back to Implement.

7. Debug

- **Use it only when something fails:** a build or test failure, an exception, wrong behaviour or a reported bug. It is not a step after every change.
- Start with evidence: repro steps or issue link, expected vs. actual behaviour, and **paste the exact error message, stack trace or log lines** as they appear, not a description from memory.
- Prefer a **targeted fix at the failure point** over broad refactoring, add a regression test, and remove temporary logging and breakpoints before committing. Then go back to Check / Review.
- Debug against local or test data. Use production data only when the Company/Client has approved it, and mask sensitive values.

8. Common rules across all stages

Rule	What it means
One task per chat	Start a new chat for each task. Long, mixed chats lose context and cost more tokens.
Project instructions	Keep project conventions in the tool's instruction file (`CLAUDE.md`, Cursor rules, `.github/copilot-instructions.md`, `AGENTS.md`, `GEMINI.md`) so every stage follows them.
Model choice	Model Selection Standard: select the lowest suitable model explicitly (not Auto); stronger for complex or critical work.
Developer owns the code	Review every suggestion and diff. Build, run tests, and go through code review as usual.
Data safety	**Never** paste secrets, credentials, production, or restricted data into chat. Use only Company/Client-approved accounts and tools.
Human approval	**Mandatory** for security, authentication/authorization, production data, infrastructure, and other high-impact changes.

9. Copy/paste prompt examples

Stage	Prompt
Understand / Explore	Explain how duplicate customers are detected in CustomerImportService, and which methods would need to change to match on email instead of customer code. Do not change any code.
Plan	Plan CSV customer import using the existing API/service pattern. Inspect the existing Customer and import code first. List the files to change, the order of steps, edge cases, and the tests to add. Ask me if anything is unclear.
Implement	Implement step 1 of the approved plan only. Follow the existing code style and do not change anything outside the plan. Then stop, show me the diff and run the tests.

Debug	Customer CSV import fails for files over 10,000 rows. Expected: the import completes and reports failed rows. Actual: it stops after about 30 seconds with this error:
	`System.TimeoutException: The operation has timed out.`
	`at CustomerImportService.ImportAsync(Stream file) in ImportService.cs:line 87`
	Reproduce it, find the root cause, and propose a targeted fix. Do not refactor unrelated code.

References:

Understand: Claude Code • Copilot in VS Code • Copilot in Visual Studio • Codex (/permissions)

Plan: Claude Code • Cursor • Copilot in VS Code • Copilot in Visual Studio • Codex (/plan) • Gemini CLI

Debug: Claude Code • Cursor • Copilot in VS Code • Copilot in Visual Studio

— End of Document —